RMTC 机甲大师图灵对决赛通信协议v1.1

更新内容：

RMTC 选手通信协议

1. 接入要求

1.1 示例选手构建与启动

1.2 双向通信确认

2. 公开话题

队伍注册 /team_X/registration

队伍名录 /referee/team_registry

3. 感知 /team_X/perception

感知示例

字段说明

感知解析要求

本车绝对坐标与视野相对位置图

开局与复活定位示例

公开事件 /referee/event_text

任务公告 /referee/mission

3.1 任务类型

4. 动作 /team_X/action

Tick 与场次对齐

错误码

无效动作计数

5. LLM 网关（OpenAI 兼容）

请求与身份

响应与错误

请求频率与配额

调用资格确认

6. 信息权限与合规要求

7. 联调与正式成绩

7.1 联调方式

7.2 成绩确认

7.3 公开测试端与正式接入

8. 场次配置与事件输入

更新内容：

- **第3节「感知」**：补充 `score.effective_damage` 的含义：实际攻击伤害＋击杀奖励－死亡扣除，最低0，可下降、可超过500；达到500时对抗项满分15分。
- **第3节「感知」**：明确普通攻击、反伤和辣椒击杀可以分配奖励；爆炸、世界分裂等环境死亡不发奖励。
- **第3节「感知」**：增加计分字段 JSON 片段和数值示例：80＋20＋60＝160，随后死亡一次降至100。
- **§3.1「任务类型」**：明确伤害任务使用调整后的有效伤害；已完成任务不因数值下降而撤销，只对结算时存活车辆检查。
- **第4节「Tick 与场次对齐」**：补充先累计伤害和奖励、再扣死亡惩罚、最后检查任务的顺序；普通战斗已经击毁的车辆不重复产生辣椒击杀奖励。

RMTC 选手通信协议

本协议规定参赛程序的运行环境、话题权限、消息格式、动作时序及统一 AI 网关接口。参赛队伍须按本协议完成接入与赛前联调。

配套赛事规则见 rules.txt。比赛权限、计分与晋级以赛事规则为准。

1. 接入要求

- 参赛程序须运行在 ROS 2 Humble 环境，`X` 为裁判分配的本场 A/B/C/D 席位；
- 订阅 `/team_X/perception` 获取感知，向 `/team_X/action` 发布动作；动作必须原样回填该感知的 `tick` 和 `session_id`；
- 统一 LLM 网关可用于高层决策；正式第二阶段每场须满足本场真实成功调用要求；
- **网关只做转发与限额，不提供任何决策逻辑。**提示词、模型输出解析、动作合法性判断、失败回退、何时调用，由参赛队伍自行实现。

赛前由裁判公布 ROS_DOMAIN_ID、裁判机地址、本场席位及所需 AI 接入信息。参赛设备须使用比赛有线网络，设置 ROS_LOCALHOST_ONLY=0。允许采用虚拟机，但须保证虚拟机与裁判机之间的

DDS 发现和双向通信正常，并通过实际感知与动作回执验证。

队伍长期登记 ID、队名与本场席位分别管理。正式名单及每场身份映射由裁判确认，选手登记消息不能修改正式名单或成绩归属。队名为 1–96 个 UTF-8 字节，不得重复或含换行。换场后须按裁判通知更新席位和令牌；动作身份由其发布话题所属席位确定，动作 JSON 无须携带队名。

1.1 示例选手构建与启动

以下命令在项目根目录执行。选手须使用与本协议一致的源文件构建参赛程序，确认运行的工作空间与构建目录一致。

```
source /opt/ros/humble/setup.bash
(cd contestant_ws && colcon build --symlink-install --packages-select rmtc_bot)
source contestant_ws/install/setup.bash
ros2 pkg prefix rmtc_bot

export ROS_DOMAIN_ID=27
export ROS_LOCALHOST_ONLY=0
export RMTC_TEAM_TOKEN='本场席位令牌'
./run_contestant.sh A '示例队伍' 192.168.1.100
```

示例中的域编号、席位、地址和令牌须替换为主办方发放的接入信息。AI 端口可通过 `RMTC_AI_PORT` 设置，默认 8000。程序重新构建后须重启进程； `ros2 pkg prefix rmtc_bot` 应指向预期的安装目录。自主开发的选手程序按相同公开协议接入，不要求使用示例策略。

1.2 双向通信确认

收到感知只证明接收方向可用。选手须提交一条符合当前场次与 Tick 的合法动作，并核对后续感知中的 `self.last_action` ，完成双向验证。感知解析失败、动作未发出或没有动作回执时，不能认定联调通过。报错信息和对应 Tick 应保存并向裁判说明，令牌不得写入公开日志。

2. 公开话题

所有公开 ROS 话题的消息类型均为 `std_msgs/msg/String` 。感知、动作、注册和队伍名录的 `data` 内容为 JSON 文本；任务和随机事件公告的 `data` 内容为纯文本。

下文 JSON 内容话题的示例直接展示 `data` 中的 JSON；纯文本话题使用 `{"data":"..."}` 展示 ROS 消息字段。后者不表示文本内部存在 JSON 结构，接收后应直接读取 `msg.data` 。

话题	选手权限	data 内容

`/team_X/perception`	订阅本队席位	JSON：当前感知
`/team_X/action`	发布本队席位	JSON：当前 Tick 动作
`/team_X/registration`	发布本队席位	JSON：队伍显示名称
`/referee/mission`	订阅	纯文本：自然语言任务
`/referee/event_text`	订阅	纯文本：自然语言随机事件公告
`/referee/team_registry`	订阅	JSON：席位与队名对应关系

`X` 为主办方分配的本场 A/B/C/D 席位。参赛程序只能按表中权限使用公开话题，不得读取其他席位的专属感知、向其他席位提交消息，或访问未列入公开接口的裁判、管理与观赛数据。禁止通过录屏识别、人工转述或其他渠道将观赛全图反馈给机器人。网络可达不代表获得参赛使用权限。

话题	可靠性	持久性	历史深度
感知、动作	RELIABLE	VOLATILE	1
注册、任务、队伍名录	RELIABLE	TRANSIENT_LOCAL	1
事件公告	RELIABLE	TRANSIENT_LOCAL	128

选手应及时处理最新感知，不批量补发历史动作。注册发布者须保持运行，以便裁判接入时接收其保留的注册消息。

队伍注册 `/team_X/registration`

向本场 A 席位注册时，`/team_A/registration` 的 `data` 内容示例：

```
1  {"team":"A","name":"星火队"}
```

队伍须向主办方分配的席位话题注册。`name` 为显示名称，UTF-8 编码长度为 1–96 字节，不得包含换行或回车。队名不用于动作路由、场次校验或 AI 鉴权；填写 `team` 字段不能取得其他席位的权限。

队伍名录 `/referee/team_registry`

`data` 内容示例：

```
{
  "teams": [
    {"team":"A","name":"星火队"},
    {"team":"B","name":"远航队"},
    {"team":"C","name":"逐光队"},
    {"team":"D","name":"凌云队"}
  ]
}
```


名录提供本场席位与显示名称的对应关系，实际条目数量取决于本场席位安排。参赛程序只能订阅该话题，不得向其发布消息。

3. 感知 `/team_X/perception`

感知示例

以下为 `/team_A/perception` 的 `data` 内容。地图、任务和数值仅用于说明消息格式。

```
{
  "tick": 148,
  "session_id": "example-match",
  "vision_radius": 4,
  "self": {"team": "A", "position": [8, 8], "hp": 70, "alive": true, "defending": false,
           "respawn_in": 0, "protected": false, "robot_id": "A",
           "last_action": {"tick": 147, "action": "move", "moved": true, "effective_direction": "right"}},
  "visible_entities": [
    {"type": "robot", "id": "robot_B", "team": "B", "robot_id": "B", "position": "000000000\n000000000\n000000000\n000000000\n000010000\n000002000\n000000000\n000000000\n000000000",
     "hp": 60, "defending": false, "protected": false},
    {"type": "resource_normal", "id": "n09", "position": "000000000\n000000000\n000000000\n000000000\n000010000\n000020000\n000000000\n000000000\n000000000", "value": 10},
    {"type": "resource_high", "id": "h01", "position": "000000000\n000000000\n000000000\n000000200\n000010000\n000000000\n000000000\n000000000\n000000000", "value": 20},
    {"type": "obstacle", "position": "000000000\n000000000\n000000000\n000000000\n000010000\n000000020\n000000000\n000000000\n000000000"}
  ],
  "score": {"resource_value": 30, "effective_damage": 0, "mission_completed": 1,
            "mission_completed_ids": ["collect_high_value"], "invalid_actions": 0},
  "mission_text": "获得至少两个普通资源；获得至少一个高价值资源；比赛结束前进入坐标 [12, 12]。",
  "missions": [
    {"id": "collect_two_normal", "type": "normal_resources", "target": 2},
    {"id": "collect_high_value", "type": "high_resources", "target": 1},
    {"id": "reach_target", "type": "reach", "target": 1, "position": [12, 12]}
  ],
  "map": {"width": 24, "height": 24},
  "event_text": ""
}
```


字段说明

字段	含义

`tick`	本条感知对应的动作 Tick，类型为整数；动作必须回填同一个值
`session_id`	本场比赛 ID，类型为字符串；动作必须原样回填，不能沿用上一场 ID
`self.team` / `self.robot_id`	本车席位及车辆标识
`self.position`	每 Tick 提供本车当前绝对坐标 `[x,y]`；复活格不一定是原出生点
`self.position_reason`	仅开局或重生首帧提供 initial 或 respawn；其余帧省略，position 始终存在
`self.last_action`	上一 Tick 的无坐标回执；开局为 null，成功移动给出 moved=true 和 effective_direction
`self.hp` / `self.alive`	生命值 / 是否存活；阵亡期间只能 `wait`
`self.defending`	最近状态的防御标记，不保证即将结算的 Tick 继续防御
`self.respawn_in`	距离复活还有多少 Tick（存活时为 0）
`self.protected`	是否处于出生保护（保护期内不能攻击）
`vision_radius`	当前规则配置的视野半径，整数 0–4；实体按横纵坐标差均不超过该值筛选
`visible_entities[]`	以本车为中心、横纵最多各 ±4 格（9×9）范围内的可见实体；实际范围可按 `vision_radius` 缩小
`visible_entities[].position`	九行字符串，以换行符分隔（JSON 文本中写作 `\n`），每行九个字符；`1` 固定在第 5 行第 5 列，`2` 为该物体，`0` 为占位
`type=robot`	其他存活机器人：`id`、`team`、`robot_id`、`position`、`hp`、`defending`、`protected`
`type=resource_normal` / `resource_high`	资源：`id`、`position`、`value`；`value >= 20` 记为 `resource_high`
`type=obstacle`	障碍：`position`

`score.*`	本队累计：资源价值、有效伤害、已完成任务数、已完成任务 ID、非法动作数
`mission_text` / `missions[]`	自然语言任务与结构化子任务；`type` 取值见 §3.1
`event_text`	本场近期自然语言公告文本，多条公告以换行分隔；须通过大语言模型理解随机事件；无公告时为空字符串
`map`	仅地图尺寸；障碍与资源的全局分布不在感知中

`visible_entities` 提供本队视野内实体，阵亡时为空。可以保留此前合法感知的记忆和推断，但不保证是实时状态；公开事件公告是额外的合法全局信息来源。`vision_radius` 是公开规则参数，不包含隐藏实体或地图真值。

`score.effective_damage` 为有效伤害：每 Tick 累计实际攻击伤害及击杀奖励后，每次死亡扣除 60，最低为 0；可为小数，可下降，可超过 500，用于对抗计分、同分排名及伤害任务判定；达到 500 时对抗项取得满分 15 分。击杀奖励在实际攻击伤害之外另行计入，普通攻击、反伤或辣椒致死时，每名被击毁车辆的击杀奖励总量为60，由对应致死批次中的合格贡献者平分。爆炸、世界分裂等其他环境死亡不发放击杀奖励。反伤和辣椒按赛事规则确定贡献者。程序应直接使用最新感知中的该字段，不应将其视为只增不减的原始伤害计数，也不应在该值上自行再次加减奖励或死亡扣除。

以下仅展示计分字段片段，不是完整感知消息

```
{"score":{"effective_damage":160.0}}
```

> 例如，此前累计值为 80，本 Tick 实际造成 20 点攻击伤害并独自取得一次击杀奖励 60，结算值为 160。随后某 Tick 无新增伤害或奖励而死亡一次，结算值为 100。

感知解析要求

只将本协议列出的公开字段作为输入要求，不得要求消息包含结构化随机事件参数、全车状态或车辆效果对象。随机事件输入为 `event_text` 字符串，无公告时为空字符串；任务的 `missions` 数组与随机事件文本分别处理。

`self.position` 为绝对坐标数组，`visible_entities[].position` 为九行相对位置字符串，不能使用同一种类型解析两者。接收端须检查字段类型并处理解析异常，不得因字段缺失或类型错误导致进程退出；无法形成可靠决策时不提交臆测动作。

本车绝对坐标与视野相对位置图

每帧以 self.position 作为本车位置；同 Tick 重发不重复推进策略，漏帧、晚加入或重启后可用下一份感知重新定位。last_action 用于解释上一 Tick 的执行结果。视野实体 position 是固定世界轴下的 9×9 相对位置字符串。动作须匹配当前 tick 和 session_id。

每个可见实体各有一张 `position` 图（JSON 字符串，非数字数组，也不是全部实体合并后的视野图），以原地图轴为基准，x 向右、y 向下，不随移动方向旋转。例如：

```
000000000
000000000
000002000
000000000
000010000
000000000
000000000
000000000
000000000
```

字符串解析后含 81 个格子字符及 8 个换行符，共 89 个字符，无末尾换行；不得将换行误读为反斜杠和字母 n 两个字符。

此例物体位于自身右 1 格、上 2 格，即相对位移 `[1,-2]`。以从 1 开始的行列编号计，位移为 `[2所在列-5,2所在行-5]`。自身与物体重叠时中心保留 `1`，图中没有 `2`，表示 `[0,0]`。地图边缘保留完整 9×9 图；`0` 不表示该格可通行或在地图内。vision_radius 仅允许整数 0–4；半径较小时按较小半径筛选，字符串为 9×9。参考节点在本地用本 Tick self.position 加相对位移，恢复规划器所需坐标。

开局与复活定位示例

以下仅展示 `self` 中与定位有关的片段：

```
{"position": [1, 1], "position_reason": "initial", "last_action": null}
```

```
{"position": [1, 2], "position_reason": "respawn"}
```

第二例表示实际复活在 `[1,2]`，不是承诺固定复活于该坐标。若动作 Tick N 的结算中发生复活，则在状态 Tick N+1 提供重生定位；世界分裂强制复活也遵循此发布时序。同一定位 Tick 的重复消息可含该字段，按 tick 去重，而不是按接收次数计数。

公开事件 `/referee/event_text`

以下为 ROS 消息字段示例，`data` 内容为纯文本：

```
{
  "data": "场次 example-match，第 50 Tick：随机事件预告：25 Tick 后（Tick 75），左上角 [4,4]、右下角 [9,9] 的矩形区域将发生爆炸，范围内所有存活战车扣除 80 HP；出生保护免疫。"
}
```

相同公告在感知中的表示示例，以下仅展示相关字段：

```
{
  "tick": 50,
  "session_id": "example-match",
  "event_text": "第 50 Tick：随机事件预告：25 Tick 后（Tick 75），左上角 [4,4]、右下角 [9,9] 的矩形区域将发生爆炸，范围内所有存活战车扣除 80 HP；出生保护免疫。"
}
```

示例用于说明文本承载方式，不代表正式比赛事件安排，也不构成可依赖的固定句式。事件话题中的 `msg.data` 是纯文本，不能按事件参数 JSON 对象解析。

参赛程序须将随机事件文本交由官方 AI 服务中的大语言模型理解，再校验模型输出并执行自主决策。禁止使用正则表达式、关键词匹配、固定句式模板或预设事件时间表直接解析随机事件并替代大语言模型判断。

公告话题逐条发送，感知中的 `event_text` 可以重复包含近期公告。重复收到公告不表示事件重复触发；同一 Tick 也可能有多条不同公告，不得仅按 Tick 去除不同内容。

公告缓存有容量限制，不保证包含整场全部历史。参赛程序应及时接收并保存本场必要信息，切换场次时清空公告记忆。不能假定重连后可以恢复所有已错过的事件。

公告可描述事件预告、生效、效果授予及结束等信息，但不保证逐项公布每辆车的全部状态变化。以当前合法感知和大语言模型对公告的理解共同形成决策依据。事件及效果的有效期包含生效 Tick，不包含结束 Tick。

任务公告 `/referee/mission`

以下为 ROS 消息字段示例：

```
{"data":"获得至少两个普通资源；获得至少一个高价值资源；比赛结束前进入坐标 [12, 12]。"}
```

`msg.data` 为任务纯文本。结构化任务描述通过本队感知中的 `missions` 获取。任务数据与随机事件公告是不同的信息类别，随机事件须遵守大语言模型解析要求。

3.1 任务类型

`type`	含义	附带字段
`normal_resources`	普通资源数量	`target`
`high_resources`	高价值资源数量	`target`
`resource_value`	资源总价值	`target`
`effective_damage`	累计有效伤害达到指定值，包含击杀奖励及死亡扣除	`target`
`reach`	至少一次到达指定坐标	`position`；不得将 `target` 解读为可重复得分次数

每个任务 ID 只结算一次；`mission_text` 是同一批任务的自然语言描述。

> 伤害任务使用本 Tick 奖励和死亡扣除完成后的有效伤害判断。已完成任务不因数值随后下降而撤销，其任务 ID 保留在 `mission_completed_ids` 中。任务检查仅对结算时存活的车辆执行；任务计分资格按赛事规则判定。

4. 动作 `/team_X/action`

以下各示例为 `/team_A/action` 的一条独立消息的 `data` 内容，不是同一 Tick 应连续发送的动作序列。

移动：

```
{"tick": 148, "session_id": "example-match", "action": "move", "direction": "right"}
```

攻击：

```
{"tick":148,"session_id":"example-match","action":"attack","direction":"up"}
```

防御：

```
{"tick":148,"session_id":"example-match","action":"defend"}
```

等待：

```
{"tick":148,"session_id":"example-match","action":"wait"}
```

- `tick`：整数，必须等于所依据感知的 `tick`；
- `session_id`：字符串，必须等于所依据感知的 `session_id`；
- `action`：`move`、`attack`、`defend`、`wait`；
- `direction`：`up`、`down`、`left`、`right`，`move` 与 `attack` 必需；
- 坐标原点 `[0,0]` 在左上角，`x` 向右增大、`y` 向下增大；
- 每 Tick 最多执行一个动作；可在截止前多次提交，保留最后一条格式、方向、Tick 和场次合规的候选，再校验车辆状态及阶段限制；
- 后到的坏格式、错误 Tick 或错误场次消息不覆盖已有候选。后到的格式正确但状态不允许的动作会覆盖前者并被拒绝，不回退到更早动作；
- 无候选或候选被拒绝时按 `wait` 处理。暂停期间不接收动作，恢复后按重新发布的感知提交。

单条动作消息上限为 4096 字节；每队每 Tick 最多处理 128 条消息，超出条数直接忽略。参赛程序应优先发送一次经校验的当前动作，避免消息积压。

Tick 与场次对齐

正式第二阶段采用 500 ms/Tick、500 Tick。每 Tick 先按移动前的位置同时结算攻击伤害及死亡，再执行存活车辆的移动。每队每 Tick 只执行一个动作；被攻击击毁的车辆不能执行本 Tick 移动；该移动回执为 `moved=false`、`error=E_STATE`，不增加无效动作次数。

> 本 Tick 的实际攻击伤害和击杀奖励全部计入后，再按死亡次数扣除有效伤害，最低为 0；随后检查存活车辆的任务。普通攻击已击毁的车辆不会再次产生本 Tick 的辣椒击杀奖励。

裁判收到状态 Tick N 后，为各队发布含 `tick=N` 和本场 `session_id` 的感知，并开始完整动作窗口。选手收到后计算动作并回填这两个字段。截止时裁判固化动作；动作窗口关闭后至下一窗口开放前、暂停期间及比赛结束后收到的动作均不接收，不会补执行到后续 Tick。

同一场次只处理最新 Tick；收到早于当前 Tick 的感知应丢弃。恢复时可能重发相同 Tick，可重新提交缓存动作，但不能重复推进规划器或位置记录。收到新的 `session_id` 时清空上一场的规划、定位和动作缓存；异步 AI 或其他后台任务中属于其他场次或早于当前 Tick 的结果不得直接作为当前动作发布。

`session_id` 从当前感知获取并原样回填，参赛程序不得自行推测或生成。切换场次时须清空上一场的动作与规划缓存；异步任务结果须核验所属场次及当前适用性。场次标识不能代替席位权限或 AI 令牌鉴权。

错误码

开放动作窗口内被拒绝的候选会记录错误码；窗口关闭、暂停或比赛结束后直接忽略的消息不另计动作错误：

码	含义
`E_PARSE`	消息不是合法 JSON
`E_SCHEMA`	`tick` / `action` 类型不符或缺失、必需的 `direction` 缺失或类型不符、动作/方向非法，或场次校验时 `session_id` 不是字符串
`E_TICK`	`tick` 与当前 Tick 不一致（常见于误用上一 Tick 的感知）
`E_SESSION`	非空场次下 `session_id` 缺失或与当前场次不一致
`E_STATE`	车辆状态或阶段不允许：阵亡、出生保护期内攻击、阶段不允许该动作
`E_RATE`	单条动作消息超过 4096 字节
`E_OOB` / `E_BLOCKED`	移动执行越界、遇障碍、占位或争格失败；通过执行回执提供，不增加无效动作计数

无效动作计数

计分口径：

- `score.invalid_actions` 按裁判最终结算的拒绝记录累加，不按每条坏消息累加；未提交不增加；阵亡时 WAIT 合法，移动/攻击/防御按 E_STATE 拒绝并在结算时增加一次；事件强制 WAIT 本身不增加；
- 动作合法但执行失败（撞墙、撞障碍、走出地图）属于 `move_failed`，不计入 `invalid_actions`，机器人原地不动。

5. LLM 网关（OpenAI 兼容）

赛事方提供统一网关地址 `<GATEWAY>`（现场公布，例如 `http://<裁判机IP>:8000`）。
它是**唯一允许使用的外部 AI 通道**。

端点	方法	说明
`<GATEWAY>/healthz`	GET	服务可用性检查
`<GATEWAY>/v1/chat/completions`	POST	OpenAI 兼容对话接口

参赛程序须使用上述公开端点。网关不提供选手配额查询或管理接口。

请求与身份

请求头：`Content-Type: application/json`、`Authorization: Bearer <本队令牌>`（必填）。令牌由赛事方按队单独发放，仅限本队使用，禁止向其他队伍披露或写入公开仓库。鉴权以 Authorization 请求头为准。每场重新签发席位令牌，其他场次令牌失效；比赛期间须使用裁判发放的本场令牌。

请求体与 OpenAI 兼容；`messages` 必须是非空数组。未提供有效 `model` 时使用主办方默认模型。参赛程序须按赛前公布的模型和输出限制设置 `model`、`max_tokens`。请求体上限为 2097152 字节（2 MiB），超过返回 413；请用 `Content-Length` 发送，带 `Transfer-Encoding` 的请求返回 501。响应以完整内容返回，不提供逐 token 增量输出。

```bash
curl -s <GATEWAY>/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <本队令牌>' \
  -d '{"messages":[{"role":"user","content":"用一句话说明采集目标"}]}'
```

```python
import json, urllib.request

TEAM_TOKEN = "<本队令牌>"

def ask(prompt):
    request = urllib.request.Request(
        "<GATEWAY>/v1/chat/completions",
        data=json.dumps({"messages": [{"role": "user", "content": prompt}]
}).encode(),
        headers={"Content-Type": "application/json",
                 "Authorization": "Bearer " + TEAM_TOKEN})
    with urllib.request.urlopen(request, timeout=1.9) as response:
        return json.loads(response.read())["choices"][0]["message"]["conte
nt"]
```

响应与错误

响应为 Chat Completions JSON，整体缓冲后返回。网关不提供剩余额度查询或响应头；额度耗尽通过 HTTP 429 及 `quota_exceeded` 错误码识别。示例为同步调用，实际机器人宜异步请求并设置本地降级，不能让模型等待阻塞动作窗口。

服务错误响应示例：

```
{
  "error": {
    "message": "Missing bearer token",
    "type": "authentication_error",
    "param": null,
    "code": "missing_token"
  }
}
```

示例用于说明响应结构，错误提示文字不作为程序判断依据。服务可能返回非 JSON 错误正文；网络断开或读取超时也可能没有完整响应。参赛程序须同时处理 HTTP 状态码、错误码、正文解析失败与连接异常。

本地错误码如下：

HTTP	type	code	触发条件
401	`authentication_error`	`missing_token`	缺少 `Authorization: Bearer <令牌>`
401	`authentication_error`	`invalid_token`	令牌未知或已被吊销
404	`invalid_request_error`	`not_found`	请求路径未被服务识别
400	`invalid_request_error`	`invalid_content_length` / `incomplete_body`	请求长度非法、重复或请求体不完整
413	`invalid_request_error`	`payload_too_large`	请求体超过 2097152 字节
429	`insufficient_quota`	`quota_exceeded`	本席位令牌额度已用尽

429	`rate_limit_error`	`rate_limit` / `concurrency_limit`	请求频率或在途请求数超限，可退避后重试
503	`server_error`	`audit_unavailable`	服务无法确认调用记录，联系裁判复核
501	`invalid_request_error`	`unsupported_transfer_encoding`	请求带 `Transfer-Encoding`
502	`upstream_error`	`upstream_unavailable`	连不上上游
504	`upstream_error`	`upstream_timeout`	上游超时（默认 30 秒）

网关不重试上游请求。429 须结合错误码处理：`quota_exceeded` 表示额度耗尽，应降级；`rate_limit` / `concurrency_limit` 可退避后重试。503 表示服务无法确认调用记录，应保持本地决策并联系裁判。

请求频率与配额

每个来源 IP 每 10 秒最多提交 100 次 POST，每个席位令牌最多同时存在 2 个在途调用。网关请求读取超时为 5 秒；参赛程序须及时完成请求发送。共享同一来源 IP 的程序共同使用该 IP 的频率限额。

- 每个席位令牌有独立的额度，按令牌全生命周期累计、**不按时间窗重置**；**只有成功（2xx）的转发消耗额度**，被拒绝的请求与上游失败都不消耗；
- 每场上限由赛前统一公告确定，0 表示不限制；不以示例中的 100 次作为固定规则。主办方每场通过签发新席位令牌重置额度；
- 在途调用占用名额但尚未计入成功调用次数，失败即释放。成功返回后即消耗次数，模型输出不理想或客户端未及时收到，不因此退回配额；
- 额度耗尽只能通过 429 的 `quota_exceeded` 错误码感知，不提供剩余额度查询；
- 调用时机与次数由参赛程序决定；额度用尽后必须自行降级。

调用资格确认

本场 AI 调用资格以主办方确认的本场、本席位成功调用记录为准。其他场次或联调中的调用不能替代本场资格。

6. 信息权限与合规要求

- 正式比赛只能通过主办方提供的统一网关使用 AI，禁止绕过网关访问其他外部 AI 服务；公开测试端可配置选手自备模型服务，并通过测试代理验证自己的程序；
- 阶段二每场最终结算前要求至少一次属于本场、本席位的真实统一 AI 成功调用；否则任务分为 0，原始任务数也不作为同分优势，资源和对抗分保留。练习、上一场或联调调用不替代本场资格，是否成功以主办方确认的本场、本席位成功调用记录为准；
- 内部话题、地图真值与其他队伍的进程均不得访问。

7. 联调与正式成绩

7.1 联调方式

联调用于验证选手程序与赛事接口的通信及运行行为，不登记正式成绩。如需联调，参赛队伍自行启动程序，主办方公布本场席位和联调安排。联调不要求先完成第一阶段或取得正式种子分组。

阶段	用途	可选席位
stage0	快速验证感知、动作与 Tick、场次对齐	A/B/C/D 中任意 1–4 个不同席位
stage1	验证第一阶段独立采集规则	仅 A
stage2	验证完整第二阶段资源对抗与事件规则	A/B/C/D 中任意 1–4 个不同席位

Stage 0 使用快速联调地图，测试完整第二阶段规则应选择 stage2。联调 Tick 周期和长度以当次安排为准。

选手接入步骤：

1. 获取本场席位、网络参数及需要使用的 AI 服务信息。
2. 设置指定的 `ROS_DOMAIN_ID` 和 `ROS_LOCALHOST_ONLY=0`，启动参赛程序。
3. 接收本队感知，按感知中的 `tick` 和 `session_id` 提交合法动作。
4. 检查后续感知中的 `self.last_action`，确认动作得到处理。仅观察到 Tick 增长不能证明动作链路正常。
5. 测试 AI 功能时，核验调用响应、超时处理及本地降级行为。

恢复比赛可能重发同一 Tick 感知，重开场次须按新的 `session_id` 清空缓存。纯感知与动作联调不要求调用 AI。测试 AI 功能须使用主办方公布的服务。

联调第二阶段按任务完成情况展示任务得分，不以真实 AI 调用作为联调任务计分条件。联调结果和调用记录不能替代正式成绩或正式比赛 AI 资格。

缺席或断线不停止 Tick 推进，未提交动作时按 WAIT 结算；重新连接后应以最新感知恢复。全局故障由裁判暂停并保全记录，选手须遵从中止、重赛或裁定通知。

7.2 成绩确认

正式成绩以主办方公布的确认结果及裁定为准。裁判可依据有效比赛记录、处罚或复核结果赋予、修改或取消成绩，修订须记录原因并通知受影响队伍。裁定零分是有效成绩；取消登记表示该结果不能作为排名或晋级依据。

参赛队伍应按公布的有效成绩和编排参加比赛。长期登记 ID 与本场席位分别管理，接入信息以本场通知为准。成绩确认、申诉及故障处理见 赛事规则第 14 节。

7.3 公开测试端与正式接入

公开测试端提供 stage1、stage2。ROS 话题名称、消息类型、QoS、感知与动作字段遵循本协议；不提供 stage0 作为测试包启动选项。第一阶段模板使用 A 席位，默认 100 ms/Tick、100 Tick；第二阶段模板使用 A–D，默认 500 ms/Tick、500 Tick。调整练习速度后，应在默认节奏下再次验证程序。

接入项	公开测试端默认值	正式比赛
感知与动作	`/team_X/perception`、`/team_X/action`	相同，X 使用本场分配席位
场次与 Tick	从当前感知读取 `session_id`、`tick`	相同，不沿用练习场次值
ROS 域	`ROS_DOMAIN_ID=27`，可在启动时指定	使用主办方公布值
AI 完整请求地址	`http://127.0.0.1:8080/v1/chat/completions`	使用主办方发放的完整地址
AI 鉴权	`Authorization: Bearer practice-X`	`Authorization: Bearer <本场席位令牌>`
AI 模式	模拟响应或选手自备真实服务	主办方提供的真实统一 AI 服务

测试代理支持非流式 `messages` 请求，`stream` 应省略或设为 `false`。测试令牌、并发限制和错误响应仅用于开发验证，不能用于推断正式调用配额和限流；正式接口要求以本协议 AI 接口章节及赛前公告为准。请求返回后须自行解析模型结果，不能把 HTTP 成功等同于策略正确。

测试页面分别显示“模拟成功 N 次”或“LLM 成功 N 次”，按本场、席位累计，重开归零。手工测试请求和参考车请求也可能计入相应席位；验证自己的程序时应单独占用席位，并从新场次开始。练习分数不应用正式 AI 资格门槛，不能据此认定正式任务分有效。

全图观赛、练习参数编辑和网页诊断仅供人工调试，不属于参赛程序公开输入。程序完成两阶段测试后，上场须更新接入配置并完成现场双向通信与真实 AI 联调。

8. 场次配置与事件输入

第二阶段不同大轮可使用不同地图，同轮各组及加赛使用同一地图。参赛程序须读取本场感知中的地图尺寸和自身坐标，不得依赖上一场缓存的地图真值。随机事件参数可在每场赛前设置，本场部署后固定；事件判断使用本场自然语言公告，不得依赖固定事件时间表。选手通过本协议列出的公开话题接入，不能获取地图真值或结构化随机事件参数。

公告中的 Tick 表示事件发生或预定生效的逻辑时刻。动作 Tick 为 N 的执行结果通过后续感知获知，不能将接收消息时刻当作事件发生时刻。消息示例中的时间、坐标和数值仅说明格式，不构成固定赛程。